

UNIVERSITÉ MOHAMMED PREMIER

École Nationale des Sciences Appliquées d'Oujda (ENSAO)

FILIÈRE GÉNIE INFORMATIQUE – GI3

RAPPORT FINAL DE MINI-PROJET

Application de Gestion de Clinique

Réalisé par :

- MAHAMAT SOUGOUMA ALI
- MARIAM EL OUALI

Encadrante :

Mme Douae EL HILA

Module :

Développement Java IHM

Année universitaire : 2025/2026

Remerciements

Nous tenons à exprimer notre profonde gratitude à notre encadrante, Madame Douae EL HILA, pour son accompagnement, ses conseils précieux et son soutien constant tout au long de la réalisation de ce mini-projet.

Ses remarques pertinentes et ses orientations rigoureuses nous ont permis d'améliorer continuellement la qualité de notre travail et d'acquérir une compréhension approfondie et pratique des technologies JavaFX, JDBC et MySQL.

Table des Matières

Remerciements

Introduction

1 Présentation du projet

1.1 Description de l'application

1.2 Besoins fonctionnels

1.3 Besoins non fonctionnels

2 Conception

2.1 Architecture MVC

2.2 Diagramme de classes UML

2.3 Diagramme DAO

2.4 Diagramme de cas d'utilisation

3 Environnement de travail

3.1 Outils et technologies utilisés

3.2 Structure du projet

4 Répartition des tâches

5 Base de données

5.1 Présentation générale

5.2 Table Patients

5.3 Table Rendez-vous

5.4 Relation et intégrité des données

6 Interfaces et Fonctionnalités

7 Tests et validation

Conclusion

Références

Introduction

Le secteur de la santé exige aujourd'hui une rigueur administrative accrue et une gestion fluide des flux de données médicales. Dans le cadre de notre formation en Génie Informatique à l'ENSAO, le module Développement Java IHM nous offre l'opportunité de lier les concepts théoriques de la programmation orientée objet à la conception d'interfaces utilisateurs riches et interconnectées avec des systèmes de gestion de bases de données.

Ce rapport présente la conception et la réalisation d'une Application de Gestion de Clinique. Ce système a pour but principal d'optimiser l'organisation des rendez-vous et le suivi des dossiers patients pour une structure médicale moderne, en se basant sur une architecture logicielle robuste et évolutive.

1 Présentation du projet

| 1.1 Description de l'application

L'application de gestion de clinique est une solution logicielle desktop conçue spécifiquement pour le personnel administratif et médical. Développée en JavaFX pour une expérience utilisateur moderne et fluide, elle s'appuie sur un serveur MySQL pour assurer la persistance et la sécurité des données.

Le système remplace les registres physiques ou les fichiers disparates en centralisant l'ensemble des interactions au sein d'une interface unique organisée selon l'architecture MVC (Modèle - Vue - Contrôleur), garantissant ainsi modularité et maintenabilité.

| 1.2 Besoins fonctionnels

Les fonctionnalités clés implémentées dans l'application se structurent autour de quatre axes majeurs :

Gestion des patients

- Enregistrement et ajout de nouveaux patients avec une fiche complète.
- Modification et mise à jour en temps réel des coordonnées et données médicales de base.
- Suppression sécurisée de fiches patients.
- Recherche intuitive multi-critères et consultation de la liste globale.

Gestion des rendez-vous

- Planification et ajout de rendez-vous médicaux personnalisés.

- Modification des horaires, affectation des médecins et mise à jour des statuts.
- Suppression ou annulation de créneaux.
- Association automatique et intègre de chaque rendez-vous à un patient unique.

Statistiques & Pilotage

- Affichage dynamique du nombre total de patients enregistrés et actifs.
- Indicateurs sur le volume global de rendez-vous.
- Segmentation et filtres des rendez-vous selon leur statut (confirmé, en attente, annulé).

Exportation de données

- Module d'extraction de la liste des patients au format standardisé CSV pour des analyses externes.

1.3 Besoins non fonctionnels

- **Ergonomie** : Interface utilisateur épurée, fluide et hautement intuitive (conçue via Scene Builder et stylisée en CSS).
- **Intégrité et Sécurité** : Protection et cohérence des données lors des transactions avec la base de données.
- **Performance** : Temps de réponse instantanés pour les requêtes de filtrage et d'extraction.
- **Architecture** : Respect strict du patron de conception MVC pour isoler la logique métier des composants graphiques.
- **Connectivité** : Driver JDBC MySQL robuste gérant efficacement le pool de connexions.

2 Conception

2.1 Architecture MVC

Pour assurer une séparation claire des responsabilités, l'application est découpée comme suit :

- **Modèle (Model)** : Représente les données et la logique métier. Classes principales : `Patient.java` et `RendezVous.java`.
- **Vue (View)** : Gère l'affichage graphique. Définie à travers les fichiers FXML : `Main.fxml`, `Patient.fxml`, `RendezVous.fxml`, et `Statistiques.fxml`.
- **Contrôleur (Controller)** : Intercepte les actions de l'utilisateur sur la Vue et met à jour le Modèle en conséquence. Classes : `PatientController.java`, `RendezVousController.java`, etc.

2.2 Diagramme de classes UML

Note de conception : Le diagramme de classes structure les données autour de deux entités majeures en forte corrélation.

- **Entité Patient** : Attributs de structure : id, nom, prenom, dateNaissance, sexe, telephone, email, adresse, groupeSanguin, actif.
- **Entité RendezVous** : Attributs de structure : id, patientId, medecin, specialite, dateRdv, heureRdv, statut, notes, prix.
- **Relation cardinale** : La relation est de type Patient (1) → (0..*) RendezVous. Un patient peut détenir plusieurs rendez-vous au fil du temps, tandis qu'un rendez-vous est obligatoirement rattaché à un et un seul patient.

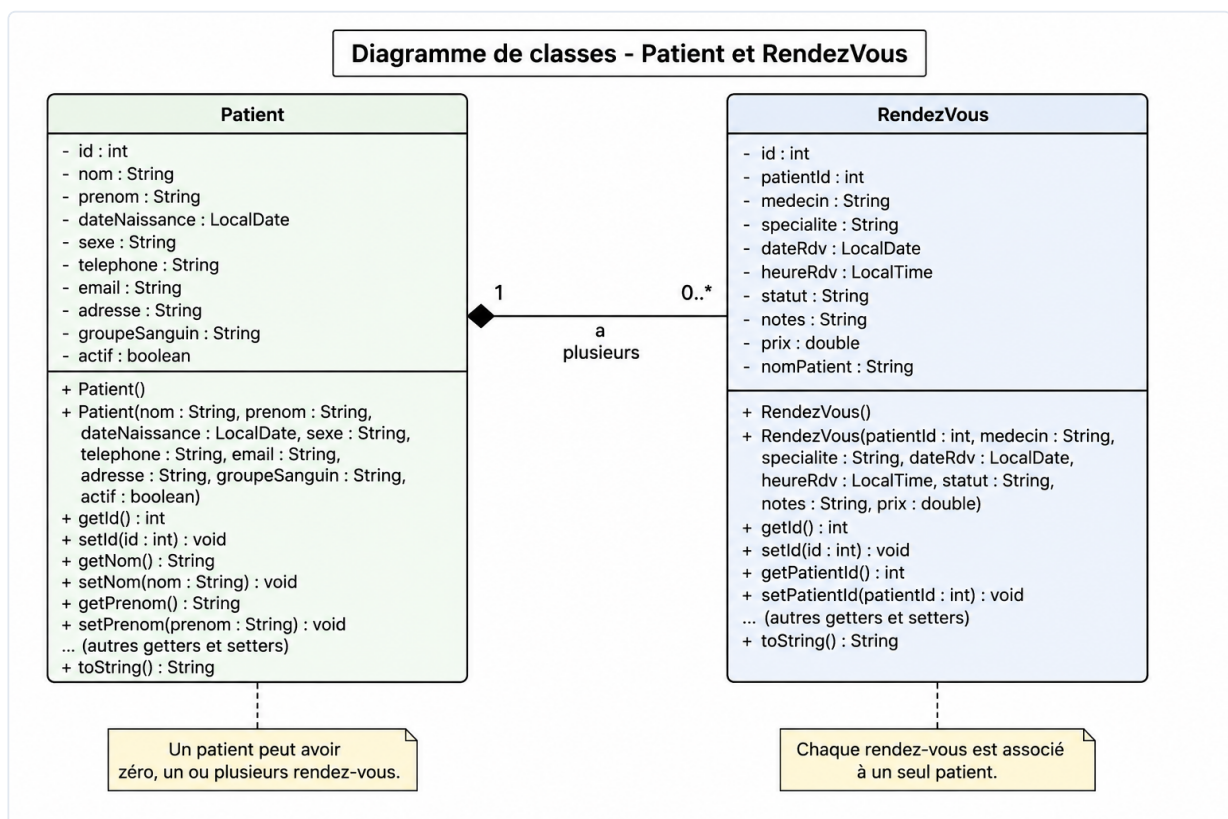


Figure 1 : Diagramme de classes du Modèle (Patient et Rendez-vous)

2.3 Diagramme DAO (Data Access Object)

Le patron d'architecture DAO a été implémenté pour isoler la logique d'accès aux données de la logique métier de l'application.

- **Classe Database** : Centralise la configuration et la gestion des connexions à la source de données MySQL. Elle intègre les attributs de configuration (URL, USER, PASSWORD) et expose la méthode critique `getConnection()` qui distribue un objet `Connection` actif.

- **PatientDAO** : Prend en charge l'ensemble des opérations CRUD dédiées à l'entité Patient : `ajouter()`, `modifier()`, `supprimer()`, et `getAll()`, ainsi que les méthodes de calcul statistique `compterTous()` et `compterActifs()`.
- **RendezVousDAO** : Prend en charge les opérations CRUD pour la table des rendez-vous et intègre des fonctions analytiques métiers telles que `compterParStatut()`.

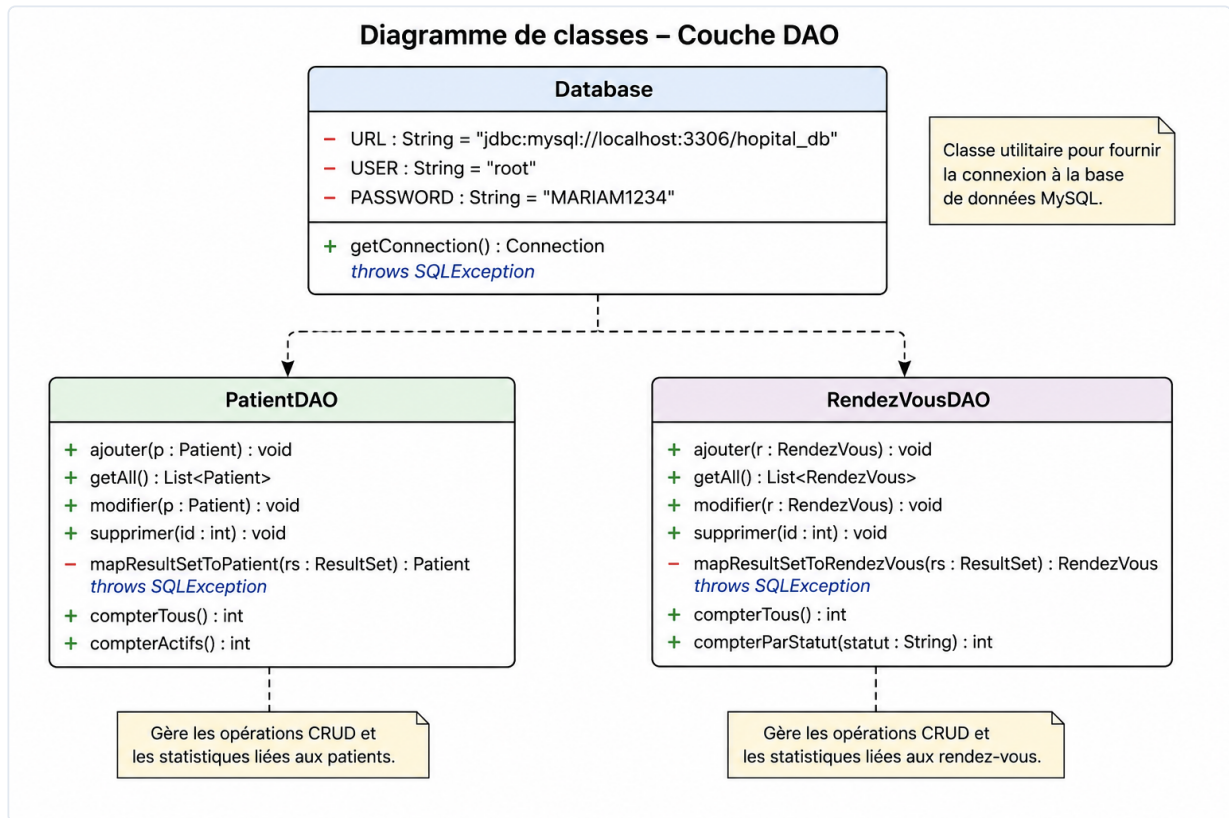


Figure 2 : Diagramme de la Couche d'Accès aux Données (DAO)

2.4 Diagramme de cas d'utilisation

Le système s'organise autour d'un acteur clé unique assurant la gestion opérationnelle de la clinique.

Acteur Principal	Catégorie de Cas d'Utilisation	Actions Métiers Disponibles
Secrétaire / Personnel Administratif	Gestion des Patients	Ajouter, Modifier, Supprimer, Rechercher multi-critères, Exporter au format CSV
	Gestion des Rendez-vous	Planifier un rendez-vous, Modifier un créneau, Annuler/Supprimer un rendez-vous
	Analyse & Suivi	Consulter le tableau de bord et les statistiques globales de fréquentation

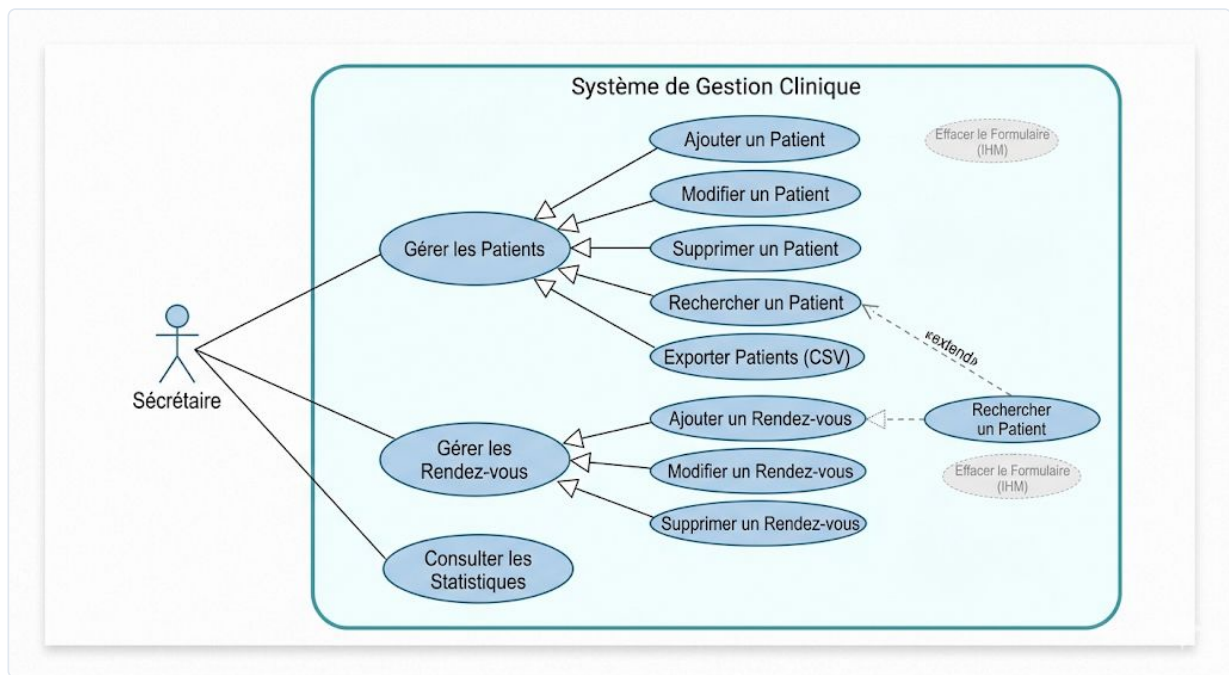


Figure 3 : Diagramme de Cas d'Utilisation (Système de Gestion Clinique)

3 Environnement de travail

3.1 Outils et technologies utilisés

L'ensemble du projet a été développé en exploitant un écosystème d'outils professionnels garantissant rigueur et efficacité dans le cycle de développement.

Outil / Technologie	Rôle et Utilisation Spécifique
IntelliJ IDEA	Environnement de développement intégré (IDE) principal pour le codage Java.
Scene Builder	Outil graphique de conception visuelle des interfaces utilisateur complexes en FXML.
MySQL Workbench	Modélisation de la base de données, exécution des scripts d'initialisation et requêtage.
GitHub	Hébergement du code source et gestion des versions au sein du binôme.
Maven	Gestionnaire de dépendances (automatisation du cycle de vie du projet et inclusion des modules JavaFX).
Java 17 & JavaFX 17	Langage de programmation moderne de base et framework applicatif pour les interfaces riches.

3.2 Structure du projet

L'organisation du code source respecte scrupuleusement l'arborescence standardisée des projets Maven d'architecture MVC :

```
HopitalApp
├── src
│   ├── main
│   │   └── java
│   │       └── com.hopital
│   │           ├── model
│   │           │   ├── Patient.java
│   │           │   └── RendezVous.java
│   │           ├── dao
│   │           │   ├── Database.java
│   │           │   ├── PatientDAO.java
│   │           │   └── RendezVousDAO.java
│   │           ├── controller
│   │           │   ├── MainController.java
│   │           │   ├── PatientController.java
│   │           │   ├── RendezVousController.java
│   │           │   └── StatistiquesController.java
│   │           └── MainApp.java
│   └── resources
│       └── com.hopital
│           ├── view
│           │   ├── Main.fxml
│           │   ├── Patient.fxml
│           │   └── style.css
```

4 Répartition des tâches

Afin de mener à bien ce projet de manière collaborative et efficace, le travail a été équitablement réparti entre les deux membres du binôme selon des axes techniques complémentaires :

Tâches de l'Étudiant 1 : MAHAMAT SOUGOUMA ALI

Domaine d'Intervention	Fichiers et Livrables Associés
Persistence & Base de données	Conception et écriture du script d'initialisation SQL (<code>init_db.sql</code>).
Modélisation Métier	Création et implémentation des classes modèles (<code>Patient.java</code> , <code>RendezVous.java</code>).
Accès aux Données (DAO)	Développement de la logique de connexion (<code>Database.java</code>) et des objets d'accès <code>PatientDAO.java</code> et <code>RendezVousDAO.java</code> .
Algorithmique Métier	Mise en place des moteurs de recherche multi-critères et filtres avancés (DAO + Controller).
Gestion de Configuration	Configuration initiale du dépôt, rédaction du <code>README.md</code> et gestion du versioning GitHub.

Tâches de l'Étudiant 2 : MARIAM EL OUALI

Domaine d'Intervention	Fichiers et Livrables Associés
Design d'Interface UI	Conception de l'ensemble des maquettes et vues XML (fichiers <code>*.fxml</code>) via Scene Builder.
Identité Visuelle & Style	Développement de la feuille de style globale en CSS (<code>style.css</code>) pour l'harmonisation de l'IHM.
Logique d'Interface	Développement complet des contrôleurs JavaFX assurant l'interaction et la liaison de données.
Composants d'Interopérabilité	Développement de la classe utilitaire d'export de données au format de fichier <code>CSVExporter</code> .
Validation Finale	Réalisation des scénarios de test et montage de la vidéo de démonstration de l'application.

5 Base de données

5.1 Présentation générale

La persistance des données applicatives est assurée par un système de gestion de base de données relationnelle (SGBDR) MySQL. La base de données est nommée `hopital_db` et contient deux tables maîtresses hautement normalisées.

5.2 Table Patients

Cette table stocke l'état civil et les informations de contact pour chaque individu enregistré au sein de la clinique.

- **id** : Clé primaire auto-incrémentée identifiant de façon unique le patient.
- **nom & prenom** : Identité du patient (champs obligatoires).
- **actif** : Booléen permettant une désactivation logique sans suppression de l'historique médical.

```
CREATE TABLE patients (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nom VARCHAR(50) NOT NULL,  
    prenom VARCHAR(50) NOT NULL,  
    date_naissance DATE,  
    sexe VARCHAR(10),  
    telephone VARCHAR(20),  
    email VARCHAR(100),  
    adresse TEXT,  
    groupe_sanguin VARCHAR(5),  
    actif BOOLEAN DEFAULT TRUE  
);
```

5.3 Table Rendez-vous

Cette table répertorie l'ensemble des consultations médicales programmées ou passées.

```
CREATE TABLE rendez_vous (
    id INT AUTO_INCREMENT PRIMARY KEY,
    patient_id INT NOT NULL,
    medecin VARCHAR(100) NOT NULL,
    specialite VARCHAR(100),
    date_rdv DATE NOT NULL,
    heure_rdv TIME NOT NULL,
    statut VARCHAR(20) DEFAULT 'En attente',
    notes TEXT,
    prix DECIMAL(10,2),
    FOREIGN KEY (patient_id) REFERENCES patients(id) ON DELETE CASCADE
);
```

5.4 Relation et intégrité des données

Une contrainte de clé étrangère lie le champ `patient_id` de la table `rendez_vous` au champ `id` de la table `patients`.

Afin de garantir une intégrité référentielle absolue et d'éviter les données orphelines, la clause **ON DELETE CASCADE** est appliquée : ainsi, la suppression définitive d'un profil de patient entraîne automatiquement et de manière transparente la suppression de l'intégralité de son historique de rendez-vous médicaux en base.

6 Interfaces et Fonctionnalités

L'interface utilisateur de l'application a été pensée pour être ergonomique et structurée en modules distincts accessibles via une barre de navigation principale :

- **Interface de Gestion des Patients :**

- *Formulaire Patient* : Un volet de saisie complet doté de validations de champs pour empêcher la saisie de données incohérentes.
- *Moteur de Recherche* : Barre de recherche dynamique filtrant instantanément la liste lors de la saisie d'un nom ou d'un numéro de téléphone.
- *TableView* : Un tableau structuré affichant l'ensemble des patients avec possibilité de tri par colonne.

- **Interface des Rendez-vous :**

- *Planification simplifiée* : Utilisation d'un composant ComboBox intelligent lié à la table des patients pour une sélection rapide du patient concerné.
- *Composants natifs* : Intégration d'un DatePicker pour une sélection calendaire fiable de la date de consultation.

- **Tableau de Bord & Statistiques** : Visualisation en temps réel des indicateurs de performance clés (KPIs) : nombre global de patients suivis, volume de rendez-vous du jour, et graphiques de répartition selon l'état des rendez-vous (confirmés ou en attente).
- **Export de Données** : Un bouton d'action dédié permet de compiler instantanément la liste des patients affichée et de générer un fichier .csv propre, prêt à être exploité sous Microsoft Excel ou tout autre outils.

7 Tests et validation

Pour s'assurer de la stabilité générale de l'application et de la robustesse des échanges entre l'IHM et MySQL, une suite de scénarios de tests fonctionnels a été rigoureusement exécutée :

ID Test	Fonctionnalité Ciblée	Scénario Opérationnel Validé	Résultat Obtenu
TEST-01	Ajout d'un Patient	Saisie complète d'une fiche et enregistrement en base.	SUCCÈS (Inséré)
TEST-02	Modification Patient	Changement de numéro de téléphone d'un patient existant.	SUCCÈS (Mis à jour)
TEST-03	Suppression Patient	Retrait d'un patient et vérification de la cascade de ses RDV.	SUCCÈS (Cascadé)
TEST-04	Recherche Dynamique	Filtrage en temps réel dans le TableView via la barre de recherche.	SUCCÈS (Instantané)
TEST-05	Planification de RDV	Sélection d'un patient via le ComboBox et enregistrement du RDV.	SUCCÈS (Légué)
TEST-06	Extraction CSV	Génération du fichier de rapports depuis la vue Patient.	SUCCÈS (Fichier créé)

Conclusion

Ce mini-projet nous a permis de mettre en application de manière concrète l'ensemble des concepts théoriques étudiés dans le module Développement Java IHM à travers la réalisation d'une application complète et fonctionnelle de gestion de clinique. Nous avons pu appréhender les forces de l'écosystème JavaFX couplé à une base de données MySQL relationnelle via JDBC, le tout solidement orchestré par l'architecture MVC.

Les fonctionnalités développées répondent précisément aux exigences initiales de notre cahier des charges. Cette application constitue une base logicielle saine et évolutive. Elle pourra à l'avenir être enrichie de modules complémentaires, tels qu'un système d'authentification sécurisé par rôles (secrétaire, médecin, administrateur), un module de facturation avec édition de factures PDF, ou encore l'envoi automatique de rappels de rendez-vous par e-mail ou SMS aux patients.

Références

- Documentation Officielle JavaFX : <https://openjfx.io>
- Documentation MySQL Server & Workbench : <https://dev.mysql.com/doc/>
- Guide d'implémentation Java JDBC : <https://docs.oracle.com/javase/tutorial/jdbc/>
- Apache Maven Project Documentation : <https://maven.apache.org>
- JetBrains IntelliJ IDEA Center : <https://www.jetbrains.com/idea/>